REAL-TIME WORKFLOW

Git, GitHub & GitHub Actions for a Playwright Framework

Practical step-by-step guide: create a test, commit changes, push to GitHub, run Playwright tests automatically, review results, and merge.

1. Understand How the Three Tools Work Together

Git: A version-control tool used to track changes, create branches, and commit code locally.

GitHub: A remote hosting and collaboration platform used for repositories, pull requests, reviews, and team workflows.

GitHub Actions: A CI/CD automation platform that runs workflows in response to events such as pushes and pull requests.

Overall flow: Local machine (Git) → git push → GitHub repository → GitHub Actions workflow → Playwright test results.

2. Complete Real-Time Workflow

1. Create or clone the repository.
2. Create a feature branch.
3. Write or update the Playwright test.
4. Run tests locally.
5. Stage and commit changes.
6. Push the branch to GitHub.
7. Create a pull request.
8. Let GitHub Actions run the configured pipeline.
9. Review results, fix failures, and push corrections.
10. Get the required approvals and merge the pull request.

3. Step-by-Step Implementation

Step 1: Install and verify tools

Install Git, Node.js (LTS), and Visual Studio Code. Create a GitHub account if needed.

```
git --version
node --version
npm --version
```

Configure the Git identity used for commits:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
git config --global --list
```

Step 2: Create or clone the repository

Option A — The repository already exists:

```
git clone https://github.com/USERNAME/PlaywrightAutomation.git
cd PlaywrightAutomation
code .
```

Replace USERNAME with the actual GitHub username or organization.

Option B — Start a new project locally:

```
mkdir PlaywrightAutomation
cd PlaywrightAutomation
git init
git branch -M main
```

Create an empty repository on GitHub, then connect the local repository:

```
git remote add origin https://github.com/USERNAME/PlaywrightAutomation.git
git remote -v
```

For a new Playwright project, run the initializer and follow the prompts to select TypeScript:

```
npm init playwright@latest
```

If you cloned an existing framework, use its existing configuration instead of initializing another project.

Step 3: Create a feature branch

```
git switch -c feature/login-test
git branch
git status
```

A feature branch isolates your work from the stable main branch. Follow the branch-naming conventions used by your team.

Step 4: Write the Playwright automation

Example project structure:

```
PlaywrightAutomation/
├── tests/
│   └── login.spec.ts
├── playwright.config.ts
├── package.json
├── package-lock.json
└── .github/
    └── workflows/
        └── playwright.yml
```

Create tests/login.spec.ts:

```
import { test, expect } from '@playwright/test';

test('user can log in successfully', async ({ page }) => {
  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('testuser');
  await page.getByLabel('Password').fill('testpassword');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL(/dashboard/);
});
```

This is an illustrative example. Replace the URL, labels, credentials, and expected URL with values for your application. Use an approved test account and never hardcode real credentials in committed code.

For a larger framework, keep locators and page actions in Page Object Model classes and keep tests focused on the scenario and assertions.

Step 5: Execute tests locally

```
npx playwright test
npx playwright test tests/login.spec.ts
npx playwright show-report
```

Run the relevant test before committing. Investigate failures in the error message, locator behavior, test data, environment, and application behavior.

Step 6: Review and stage changes

```
git status
git diff
git add tests/login.spec.ts
git diff --staged
```

Stage only the intended files. If other required files changed, stage them too. You can use git add ., but inspect git status first.

Step 7: Commit changes

```
git commit -m "Add login automation test"
git log -1 --oneline
```

A commit records a snapshot of staged changes in the local Git repository. It does not automatically upload the changes to GitHub.

Step 8: Push the feature branch to GitHub

```
git push -u origin feature/login-test
```

git push uploads committed changes; origin is the remote repository name; feature/login-test is the branch; -u sets upstream tracking for future pushes.

Open GitHub and select the feature/login-test branch to verify that the code is present. If authentication is requested, use an approved method such as Git Credential Manager or SSH. Do not put tokens or passwords directly into commands or source code.

Step 9: Create a pull request

11. Open the repository on GitHub and select Pull requests.
12. Click New pull request.
13. Set the base branch to main and the compare branch to feature/login-test.
14. Review changed files and provide a clear title and description.
15. Create the pull request.

Example description:

```
Objective: Add login automation coverage.
Changes: Added a Playwright login test and validation for successful navigation.
Validation: Executed the test locally.
Review focus: Validate locators, assertions, and test-data handling.
```

A pull request requests review and merging; creating it does not mean the code has already been merged.

4. Configure GitHub Actions to Run Playwright Tests

Step 10: Create the workflow file

Create .github/workflows/playwright.yml:

```yaml
name: Playwright Automation Tests

on:
  push:
    branches:
      - main
      - 'feature/**'
  pull_request:
    branches:
      - main

permissions:
  contents: read

jobs:
  playwright-tests:
    name: Run Playwright Tests
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      - name: Run Playwright tests
        run: npx playwright test

      - name: Upload Playwright report
        if: ${{ always() }}
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: playwright-report/
          if-no-files-found: ignore
          retention-days: 7
```

This is a starting example. Use a Node.js version supported by your project and commit package-lock.json. The workflow assumes Playwright writes HTML reports to playwright-report/; adjust the path if your reporter configuration uses a different directory.

How the workflow configuration works

Configuration	Purpose
on.push	Starts the workflow when a matching branch receives a push.
on.pull_request	Starts the workflow for pull requests targeting main.
runs-on	Selects the runner machine.
actions/checkout	Checks out the repository code.
actions/setup-node	Sets up Node.js and npm caching.
npm ci	Installs dependencies from the lockfile.
npx playwright install --with-deps	Installs browsers and required system dependencies.
npx playwright test	Executes the automated test suite.
upload-artifact	Preserves the generated report for later inspection.

Step 11: Commit and push the workflow

```
git add .github/workflows/playwright.yml
git commit -m "Add Playwright GitHub Actions workflow"
git push
```

Because the feature branch tracks its upstream, git push normally pushes to that branch. Open the Actions tab on GitHub to see the workflow run.

Step 12: Understand what happens in the pipeline

16. Trigger received: a push or pull request matches the workflow configuration.
17. Runner starts: GitHub provides an Ubuntu runner for the job.
18. Repository and dependencies are prepared: code is checked out, Node.js is set up, npm dependencies are installed, and Playwright browsers are installed.
19. Tests execute: Playwright runs the suite and records pass/fail results.
20. Report is uploaded: the workflow attempts to preserve the HTML report as an artifact, including after a test failure.

Example: pushing feature/login-test triggers a run because feature/** matches. Opening a pull request targeting main triggers another run. These are separate events and can produce separate runs.

Step 13: Review pipeline results

21. Open the repository and select Actions.
22. Open the latest Playwright workflow run.
23. Select the Run Playwright Tests step.
24. Inspect the console output.
25. If the report artifact exists, download it and inspect the test results.

If the workflow passes, confirm the required checks and reviews before merging. If it fails, identify whether the issue is dependency installation, browser setup, application access, test data, locators, assertions, or CI configuration.

A failed workflow does not necessarily mean the application is defective. Environment issues, missing secrets, configuration errors, or flaky tests can also cause failures.

Step 14: Merge the pull request

26. Open the pull request.
27. Review the conversation and changed files.
28. Confirm required checks have passed.

29. Merge using the method allowed by repository rules.
30. Delete the feature branch if that is your team's practice.

The merge can trigger another workflow run because the workflow is configured for pushes to main.

Step 15: Update local main

```
git switch main
git pull origin main
git switch -c feature/logout-test
```

Start the next task from the latest merged code.

5. Daily Workflow Checklist

- Pull the latest code from main.
- Create a feature branch.
- Write or update the Playwright test.
- Run the test locally.
- Review and stage the changes.
- Commit with a meaningful message.
- Push the branch to GitHub.
- Create or update the pull request.
- Review GitHub Actions results.
- Fix failures and push corrections.
- Get approval and merge the pull request.

6. Common Git Commands

Command	Use
git status	Check changed and staged files.
git branch	List local branches.
git switch -c feature/name	Create and switch to a new branch.
git diff	Review unstaged changes.
git add filename	Stage a file.
git commit -m "message"	Create a commit.
git push	Push commits to the tracked remote branch.
git pull origin main	Fetch and integrate updates from main.
git log --oneline	View recent commits.
git restore filename	Discard unstaged changes in a file; use carefully.

7. Common Real-Time Failures and Solutions

Problem	What to check
git push rejected	Fetch remote changes, understand conflicts, integrate changes, and push again.
Authentication failed	Verify GitHub authentication and repository access.

npm ci failed	Check package.json and package-lock.json consistency and dependency availability.
Playwright browser missing	Ensure the workflow installs the browsers required by the project.
Passes locally but fails in CI	Check environment variables, browser differences, application availability, test data, timeouts, and test isolation.
Workflow does not start	Verify workflow path, YAML syntax, branch filters, event type, and repository Actions settings.
HTML report missing	Check the Playwright reporter configuration and report directory.

8. Best Practices

- Never commit passwords, access tokens, API keys, or production secrets.
- Keep package-lock.json under version control for reproducible npm installations.
- Use feature branches and pull requests rather than directly pushing to protected main branches.
- Keep tests independent and avoid unnecessary hard waits.
- Configure required GitHub Actions checks in branch protection or repository rules when successful tests are required before merging.
- Store CI credentials in GitHub Actions secrets or an approved secret-management system.
- A green pipeline means the configured checks passed; it does not prove that every application behavior is defect-free.

9. Interview Explanation

“In my Playwright automation project, I use Git for version control and GitHub for remote code management and collaboration. I create a feature branch, develop and validate my Playwright test locally, stage and commit the changes, and push the branch to GitHub. I then create a pull request for code review.

“GitHub Actions is configured through a YAML workflow under .github/workflows. When a matching push or pull-request event occurs, the runner checks out the repository, sets up Node.js, installs the dependencies and Playwright browsers, and executes the test suite. The results are available in the Actions logs, and the HTML report can be uploaded as an artifact.

“If the pipeline fails, I analyze the failed step, fix the underlying issue, and push the correction. After the required reviews and checks pass, the pull request is merged into main. This workflow supports collaboration, early defect detection, and continuous validation of the automation framework.”

Remember the flow: Write code → Git commit → GitHub push → Pull request → GitHub Actions test execution → Review → Merge → Update local code.